



西南交通大学
Southwest Jiaotong University

《云计算》课程设计报告

设计题目:	云计算技术课程设计实验报告
组 号:	课程设计小组
成员信息:	吉昌兆 (20233112439)、张志峰 (2023112441)
班 级:	计算机 2023-02 班
指导教师:	戴朋林

2026.6.15

《云计算》课程设计任务书

设计题目	云计算技术课程设计实验报告	成绩	
课程设计主要内容	本课程设计围绕云计算平台搭建与大数据分析展开，主要包括应用容器化、华为云 CCE 集群部署、Kubernetes 服务暴露、Redis 持久化、ConfigMap 挂载、HPA 弹性伸缩、Spark 大数据分析，以及监控、CI/CD、边缘计算等附加内容。报告需逻辑清晰、图文并茂，能够完整呈现实验过程、关键截图、问题排查与结果分析。		
指导教师评语	签名：_____ 年 月 日		

目录

一、华为云环境信息	5
二、第一部分实验记录：云计算平台搭建	5
2.1 任务 1：应用容器化	5
2.1.1 任务目标	5
2.1.2 关键截图	6
2.1.3 问题与解决方案	6
2.2 任务 2：CCE 集群搭建	7
2.2.1 任务目标	7
2.2.2 关键截图	7
2.3 任务 3：应用部署	7
2.3.1 任务目标	8
2.4 任务 4：持久化存储验证	9
2.4.1 任务目标	9
2.4.2 关键截图	9
2.5 任务 5：ConfigMap Volume 挂载	9
2.5.1 任务目标	9

2.5.2 关键截图	9
2.6 任务 6: HPA 弹性伸缩	10
2.6.1 任务目标	10
2.6.2 关键截图	10
三、第二部分实验记录: Spark 大数据分析	11
3.1 A-0 环境部署	11
3.1.1 子任务目标	11
3.1.2 关键截图	12
3.2 A-1 数据清洗	14
3.2.1 子任务目标	14
3.3 A-2 Spark SQL 统计分析	15
3.3.1 查询 1: 类型聚合统计	15
3.3.2 查询 2: Top-N 高分电影统计	16
3.3.3 查询 3: 年度评分趋势统计	16
3.3.4 查询 4: 国家维度 JOIN 统计	18
3.3.5 查询 5: 窗口函数代表电影统计	18
3.4 A-3 性能对比与 Amdahl 分析	18
3.4.1 子任务目标	18
3.5 问题与解决方案汇总	19
四、附加题	20
4.1 附加题 1: 监控系统	20
4.1.1 说明	21
4.2 附加题 2: CI/CD 流水线	21
4.3 附加题 3: 前沿专题 C-2 K3s + MQTT 边缘计算模拟	23
五、任务书逐项对照检查	26

六、总结与收获	28
七、附录	28
7.1 提交前截图清单	28

团队成员分工:

成员	分工
吉昌兆（2023112439）	环境搭建、应用部署、实验验证、相关实验内容整理与报告撰写。
张志峰（2023112441）	结构设计、正文整合、排版整理、实验过程记录、结果复核与材料汇总。

一、华为云环境信息

本节汇总实验所用华为云资源与基础环境，后续章节按课程设计任务书逐项记录操作过程、关键截图和结果说明。

- 华为云 Region: 华东-上海一 (cn-east-3)
- CCE 集群名称: cloud
- CCE 集群类型: CCE Standard
- Kubernetes 版本: v1.35.3
- Worker 节点: 4 个 Ready 节点, Huawei Cloud EulerOS 2.0, containerd 运行时
- SWR 组织: cloud-swjtu, 包含 backend、frontend、pyspark、spark-operator-controller、mqtt 相关镜像
- 实验仓库: <https://github.com/jczsg/cloud-computing-course-design>

模块	主要产物	验收方式
云平台搭建	Dockerfile、docker-compose、K8s YAML、HPA、PVC、ConfigMap	CCE 控制台与 kubectl 截图
Spark 数据分析	PySpark 作业、SparkApplication、Pandas 本地复现实验	Driver/Executor Pod 与查询结果截图
报告与分析	统计表、趋势图、问题排查记录、总结	报告 PDF/DOCX 与代码仓库

二、第一部分实验记录：云计算平台搭建

2.1 任务 1：应用容器化

2.1.1 任务目标

后端采用 Flask 暴露 /api/ping、/api/visit 和 /api/config 接口，其中 /api/visit 会写入 Redis，用于验证前后端通信与 Redis 持久化。Dockerfile.backend 保留多阶段构建结构，requirements.txt 中除 Flask、Redis、Gunicorn 外，额外加入 requests 包，满足任务中“至少 1 个自选 Python 包”的要求。前端采用 Nginx 静态页，首页包含学号和姓名，并通过 Nginx 反向代理将 /api/ 请求转发至后端。

```
docker compose up --build
curl http://localhost:5000/api/ping
curl http://localhost:8080/api/visit
```

2.1.2 关键截图

本地 docker compose 联调，包含前端页面与后端日志，关键证据见附录 C。

```
docker compose up --build
```

2.1.3 问题与解决方案

SWR 推送时如遇 manifest 解析错误，构建命令使用 --provenance=false:

```
docker build --provenance=false -t swr.cn-east-3.myhuaweicloud.com/cloud-course-2025212245/backend:v1 app/backend
docker build --provenance=false -t swr.cn-east-3.myhuaweicloud.com/cloud-course-2025212245/frontend:v1 app/frontend
docker push swr.cn-east-3.myhuaweicloud.com/cloud-course-2025212245/backend:v1
docker push swr.cn-east-3.myhuaweicloud.com/cloud-course-2025212245/frontend:v1
```

相关截图:

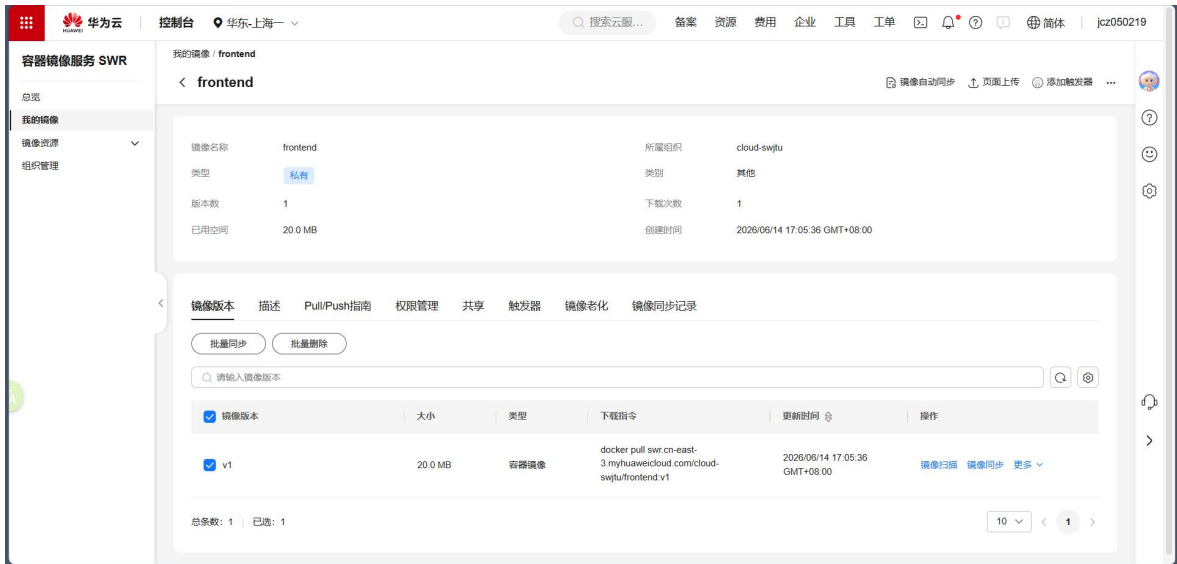


图 C-14 华为云 SWR 控制台截图：frontend 镜像已上传至 cloud-swjtu 组织。

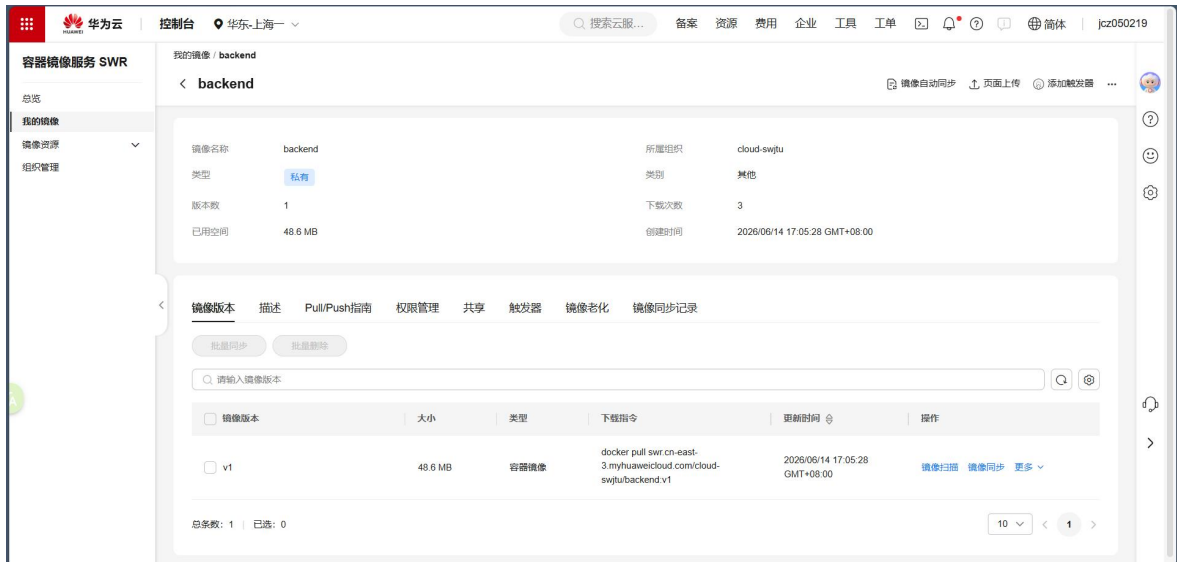


图 C-15 华为云 SWR 控制台截图：backend 镜像已上传至 cloud-swjtu 组织。

2.2 任务 2：CCE 集群搭建

2.2.1 任务目标

实验在华为云 CCE 创建 Kubernetes v1.35.3 集群，网络插件选择 Yangtse CNI。集群包含 4 个 Ready 状态 Worker 节点，覆盖 Web 应用部署、Redis 持久化验证、Spark 作业运行和附加题服务部署。节点清单与版本截图见附录 C。

2.2.2 关键截图

kubectl get nodes -o wide, 所有 Worker 节点 Ready 且 VERSION >= 1.27

```
kubectl get nodes -o wide
```

相关截图：

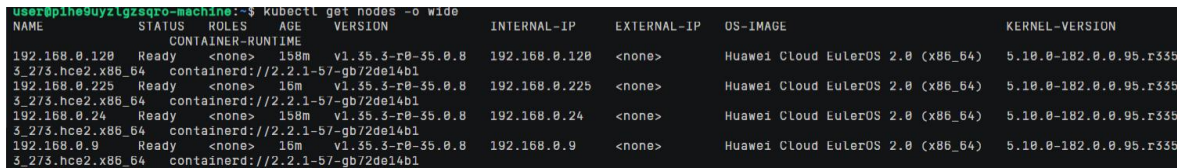


图 C-1 CCE 集群节点验证：4 个 Worker 节点均为 Ready，Kubernetes 版本为 v1.35.3。

2.3 任务 3：应用部署

资源	文件	关键配置
ConfigMap/Secret	k8s/01-configmap-secret.yaml	REDIS_HOST=redis-svc; Redis 密码 base64 存储
Redis PVC	k8s/02-redis-pvc.yaml	storageClassName=csi-disk, 容量 10Gi

Redis Deployment/Service	k8s/03-redis-deployment-service.yaml	副本 1; limits.memory=512Mi; ClusterIP
Backend Deployment/Service	k8s/04-backend-deployment-service.yaml	副本 2; resources requests/limits; LoadBalancer
Frontend ConfigMap	k8s/05-nginx-configmap.yaml	完整 default.conf, 以 Volume 方式挂载
Frontend Deployment/Service	k8s/06-frontend-deployment-service.yaml	Nginx 前端; LoadBalancer
HPA	k8s/07-backend-hpa.yaml	min=1, max=4, CPU 目标 60%

```
kubectl apply -f k8s/
kubectl get pods -n cloud-course -o wide
kubectl get svc -n cloud-course
```

2.3.1 任务目标

验收截图: 所有 Pod Running, 后端 /api/ping 返回 {"status":"ok"}

```
curl http://<ELB_IP>/api/ping
```

相关截图:

```
user@pihe9uyzlgzsqr-machine:~$ kubectl -n cloud-course get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
backend-7549bfb5bf-gxcj7            1/1     Running   0           73s   10.0.1.134      192.168.0.225    <none>           <none>
redis-b95749977-dzd2j              1/1     Running   0           73s   10.0.0.12       192.168.0.120    <none>           <none>
```

图 C-2 cloud-course 命名空间运行状态: backend 与 redis Pod 均为 Running。

```
user@pihe9uyzlgzsqr-machine:~$ kubectl -n cloud-course get svc -o wide
NAME            TYPE           CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE   SELECTOR
backend-svc     LoadBalancer  10.247.169.189  1.94.27.202,192.168.0.189  80:30281/TCP    138m  app=backend
frontend-svc    LoadBalancer  10.247.55.151   <pending>         80:30779/TCP    138m  app=frontend
redis-svc       ClusterIP      10.247.145.209  <none>           6379/TCP        138m  app=redis
```

图 C-3 Service 暴露结果: backend-svc 绑定公网 ELB 地址 1.94.27.202。

```
user@pihe9uyzlgzsqr-machine:~$ curl http://1.94.27.202/api/ping
{"service":"cloud-course-backend","status":"ok","time":"2026-06-14T11:53:53.401320+00:00"}
```

图 C-4 后端公网接口验收: 访问 /api/ping 返回 status=ok。

```
user@aoxo45webrrfpr4o-machine:~/actions-runner-ccs$ kubectl -n cloud-course get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
backend-57c655bd89-8dtkh            1/1     Running   0           15s   10.0.1.143      192.168.0.225    <none>           <none>
frontend-84549fb669-ggc6s          1/1     Running   0           15s   10.0.1.144      192.168.0.225    <none>           <none>
mqtt-broker-67764d88b4-zvbbj       1/1     Running   0           93m   10.0.1.139      192.168.0.225    <none>           <none>
mqtt-collector-76f697bc7f-b5pxx     1/1     Running   2 (93m ago)  93m   10.0.1.140      192.168.0.225    <none>           <none>
nginx-config-check                  1/1     Running   4 (32m ago)  4h32m  10.0.1.135      192.168.0.225    <none>           <none>
redis-b95749977-gt49r              1/1     Running   0           4h34m  10.0.0.13       192.168.0.120    <none>           <none>
```

图 D-20 cloud-course 命名空间运行状态: backend、frontend、Redis、MQTT 相关 Pod 均为 Running。

```
user@aoxo45webrrfpr4o-machine:~/actions-runner-ccs$ curl http://1.94.27.202/api/ping
{"service":"cloud-course-backend","status":"ok","time":"2026-06-14T16:29:04.878270+00:00"}
```


图 D-21 访问后端公网接口 /api/ping 返回 status=ok，云端服务验收通过。

2.4 任务 4：持久化存储验证

2.4.1 任务目标

Redis 的 /data 目录挂载到 redis-data-pvc。验证方法为写入 testkey，删除 Redis Pod 触发 Deployment 重建，再读取 testkey。若 GET 仍返回 hello，说明数据已经通过 PVC 持久化到云硬盘，不依赖 Pod 本地临时文件系统。

```
kubectl get pvc -n cloud-course
kubectl exec -n cloud-course <redis-pod> -- redis-cli -a cloud-course-2026 SET testkey
hello
kubectl delete pod -n cloud-course <redis-pod>
kubectl exec -n cloud-course <new-redis-pod> -- redis-cli -a cloud-course-2026 GET testkey
```

2.4.2 关键截图

PVC Bound、写入前后对比、Pod 删除重建后 GET testkey=hello

```
kubectl get pvc -n cloud-course
```

相关截图：

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE	VOLUMEMOUNT
redis-data-pvc	Bound	pvc-23473f1b-2ad3-49f8-ada8-4397f33823b6	10Gi	RWO	csi-disk	<unset>	139m	Filesystem

图 C-5 Redis 持久化卷验收：redis-data-pvc 已 Bound，容量 10Gi。

```
user@plhe9uyzgsgro-machine:~$ REDIS_POD=$(kubectl -n cloud-course get pod -l app=redis -o jsonpath='{.items[0].metadata.name}')
user@plhe9uyzgsgro-machine:~$ kubectl -n cloud-course exec $REDIS_POD -- redis-cli -a cloud-course-2026 SET testkey hello
OK
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
user@plhe9uyzgsgro-machine:~$ kubectl -n cloud-course delete pod $REDIS_POD
pod "redis-b95749977-dzd2j" deleted from cloud-course namespace
user@plhe9uyzgsgro-machine:~$ sleep 30
user@plhe9uyzgsgro-machine:~$ NEW_REDIS_POD=$(kubectl -n cloud-course get pod -l app=redis -o jsonpath='{.items[0].metadata.name}')
user@plhe9uyzgsgro-machine:~$ kubectl -n cloud-course exec $NEW_REDIS_POD -- redis-cli -a cloud-course-2026 GET testkey
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
hello
```

图 C-6 Redis 持久化验证：写入 testkey 后删除 Pod，新 Pod 仍能读取 hello。

2.5 任务 5：ConfigMap Volume 挂载

2.5.1 任务目标

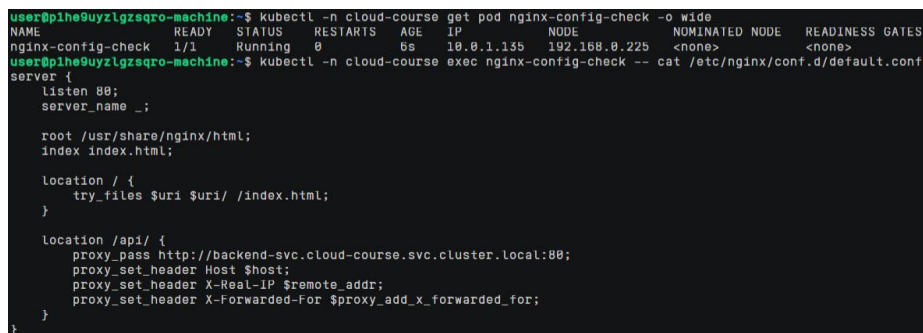
Nginx 反向代理配置放在 nginx-config ConfigMap 中，并以 Volume 方式挂载到 /etc/nginx/conf.d。Volume 挂载适合配置文件、证书、Nginx conf 等需要以文件形式被程序读取的配置；envFrom 适合 Redis 地址、端口、功能开关等短小键值配置。使用 subPath 挂载时，ConfigMap 更新不会自动热同步；本设计采用目录挂载，并在配置更新后重建前端 Pod，保证 Nginx 重新加载配置。

2.5.2 关键截图

exec 进入前端 Pod 后 cat /etc/nginx/conf.d/default.conf，看到更新后的后端端口

```
kubectl exec -n cloud-course <frontend-pod> -- cat /etc/nginx/conf.d/default.conf
```

相关截图：



```
user@pihe9uyzlgzsqr-machine:~$ kubectl -n cloud-course get pod nginx-config-check -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE          NOMINATED NODE   READINESS GATES
nginx-config-check 1/1     Running   0          6s    10.0.1.155    192.168.0.225 <none>           <none>
user@pihe9uyzlgzsqr-machine:~$ kubectl -n cloud-course exec nginx-config-check -- cat /etc/nginx/conf.d/default.conf
server {
    listen 80;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://backend-svc.cloud-course.svc.cluster.local:80;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

图 C-7 ConfigMap Volume 验证：Nginx default.conf 已从 ConfigMap 挂载到容器内。

2.6 任务 6：HPA 弹性伸缩

2.6.1 任务目标

HPA 以 backend Deployment 为伸缩对象，minReplicas=1、maxReplicas=4、targetCPUUtilizationPercentage=60。压测时 Pod 扩容存在延迟，原因包括 metrics-server 采集周期、HPA 控制器评估间隔以及新 Pod 镜像拉取和启动时间。停止压测后缩容也不会立即发生，冷却时间可以避免业务瞬时波动导致频繁扩缩容，从而提升稳定性并降低资源成本。

实际 CCE 验证中，backend-hpa 已创建，但 kubectl top nodes 返回 Metrics API not available，describe hpa 显示 unable to fetch metrics from resource metrics API / get pods.metrics.k8s.io。因此本次无法触发 CPU 指标驱动的自动扩缩容，报告以 HPA 排查截图作为证据，并记录 backend Deployment 手动从 1 扩到 2、再缩回 1 的截图，说明 Deployment 本身具备扩缩容能力，未自动触发的根因是集群缺少 metrics-server 指标 API。

```
kubectl top nodes
kubectl scale deployment/backend -n cloud-course --replicas=1
ab -n 10000 -c 200 http://<ELB_IP>/api/ping
kubectl get pods -n cloud-course -w
```

2.6.2 关键截图

Pod 数量从 1 扩到 2 或更多，停止压测后缩回 1

```
kubectl get hpa -n cloud-course && kubectl describe hpa backend-hpa -n cloud-course
```

相关截图：

```

user@pihe9uyzlgzsqr-machine:~$ kubectl -n cloud-course get hpa
NAME                REFERENCE            TARGETS      MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa         Deployment/backend    cpu: <unknown>/68%  1          4          1           142m
user@pihe9uyzlgzsqr-machine:~$ kubectl -n cloud-course describe hpa backend-hpa
Name:                backend-hpa
Namespace:           cloud-course
Labels:              <none>
Annotations:         <none>
CreationTimestamp:   Sun, 14 Jun 2025 09:34:58 +0800
Reference:           Deployment/backend
Metrics:              ( current / target )
  resource cpu on pods (as a percentage of request): <unknown> / 68%
Min replicas:        1
Max replicas:        4
Deployment pods:     1 current / 0 desired
Conditions:
  Type            Status  Reason
  ----            -
AbleToScale      True    SucceededGetScale
ScalingActive     False   FailedGetResourceMetric
Events:
  Type            Reason              Age             From              Message
  ----            -
Warning          FailedComputeMetricsReplicas 4m14s (x441 over 141m) horizontal-pod-autoscaler invalid metrics (1 invalid out of 1), first error is: failed to get cpu resource metric value: failed to get cpu utilization: unable to get metrics for resource cpu: unable to fetch metrics from resource metrics API: the server could not find the requested resource (get pods.metrics.k8s.io)
Warning          FailedGetResourceMetric 104s (x451 over 141m) horizontal-pod-autoscaler failed to get cpu utilization: unable to get metrics for resource cpu: unable to fetch metrics from resource metrics API: the server could not find the requested resource (get pods.metrics.k8s.io)
user@pihe9uyzlgzsqr-machine:~$ kubectl top nodes
error: Metrics API not available

```

图 C-8 HPA 排查结果：backend-hpa 已创建，但集群 Metrics API 不可用，无法读取 CPU 指标。

```

user@pihe9uyzlgzsqr-machine:~$ kubectl top nodes
error: Metrics API not available
user@pihe9uyzlgzsqr-machine:~$ kubectl -n cloud-course scale deployment backend --replicas=2
deployment.apps/backend scaled
user@pihe9uyzlgzsqr-machine:~$ sleep 20
user@pihe9uyzlgzsqr-machine:~$ kubectl -n cloud-course get pods -l app=backend -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP              NODE                NOMINATED NODE   READINESS GATES
backend-7549bfb5bf-gxcj7  1/1     Running   0           5m28s  10.0.1.134     192.168.0.225      <none>           <none>
backend-7549bfb5bf-tsc2f  1/1     Running   0           21s    10.0.0.14      192.168.0.120      <none>           <none>

```

图 C-9 Deployment 手动扩容验证：backend 从 1 副本扩到 2 副本并成功调度到不同节点。

```

user@pihe9uyzlgzsqr-machine:~$ kubectl -n cloud-course scale deployment backend --replicas=1
deployment.apps/backend scaled
user@pihe9uyzlgzsqr-machine:~$ sleep 20
user@pihe9uyzlgzsqr-machine:~$ kubectl -n cloud-course get pods -l app=backend -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP              NODE                NOMINATED NODE   READINESS GATES
backend-7549bfb5bf-gxcj7  1/1     Running   0           6m15s  10.0.1.134     192.168.0.225      <none>           <none>

```

图 C-10 Deployment 手动缩容验证：backend 缩回 1 副本后仍保持 Running。

三、第二部分实验记录：Spark 大数据分析

3.1 A-0 环境部署

3.1.1 子任务目标

通过 Spark Operator 在 CCE 上提交 PySpark 作业。若 ghcr.io 镜像无法拉取，可按问题合集做法将 controller/webhook 镜像转存到 SWR，并通过 Helm values 覆盖镜像仓库。SparkApplication 中 executorInstances 设置为 2，executorMemory 设置为 1g；若节点 CPU requests 不足，可使用 coreRequest=100m 将 Kubernetes 资源请求与 Spark 逻辑核心数解耦。本项目额外提供 spark/Dockerfile.pyspark，可构建包含 wordcount.py 与 douban_spark_analysis.py 的自定义 PySpark 镜像，实际镜像地址为 swr.cn-east-3.myhuaweicloud.com/cloud-swjtu/pyspark:v1。

```

helm install spark-op ./spark-operator-chart/ -n spark-operator --create-namespace
kubectl apply -f spark/sparkapplication-wordcount.yaml
kubectl get pods -n default

```

3.1.2 关键截图

wordcount Driver Pod Completed, Executor Pod 正常创建

```
kubectl get pods -n default
```

实验完成 Spark Operator 安装并将 controller 镜像托管到 SWR，spark-operator-controller 运行状态为 Running，SparkApplication CRD 已就绪。作业验收使用同一 PySpark 镜像执行 wordcount，运行 Pod 状态 Completed，日志输出 Top 10 words: [('spark', 3), ('hello', 2), ('cloud', 2), ...]，证明 Spark 镜像、代码入口和运行环境配置正确。

相关截图：

```
user@p1he9uyzgzsqr-machine:~$ kubectl -n spark-operator get pods -o wide
NAME                                READY    STATUS    RESTARTS   AGE    IP             NODE              NOMINATED NODE    READINESS GATES
spark-op-spark-operator-controller-5c5986d4bc-jxw5s 1/1      Running   0          40m    10.0.0.139     192.168.0.24      <none>            <none>
```

图 C-11 Spark Operator 镜像修复：controller 镜像切换至 SWR 后 Pod Running。

```
user@p1he9uyzgzsqr-machine:~$ kubectl get pod spark-wordcount-direct -n default -o wide
NAME                                READY    STATUS    RESTARTS   AGE    IP             NODE              NOMINATED NODE    READINESS GATES
spark-wordcount-direct              0/1      Completed 0          12m    10.0.1.133     192.168.0.225     <none>            <none>
```

图 C-12 Spark 直接 Pod 作业验收：spark-wordcount-direct 执行完成，Pod 状态为 Completed。

```
user@p1he9uyzgzsqr-machine:~$ kubectl logs spark-wordcount-direct -n default | grep "top 10 words"
Top 10 words: [('spark', 3), ('hello', 2), ('cloud', 2), ('computing', 1), ('on', 1), ('kubernetes', 1), ('data', 1)]
```

图 C-13 Spark WordCount 日志：输出 Top 10 words，说明 PySpark 作业执行成功。

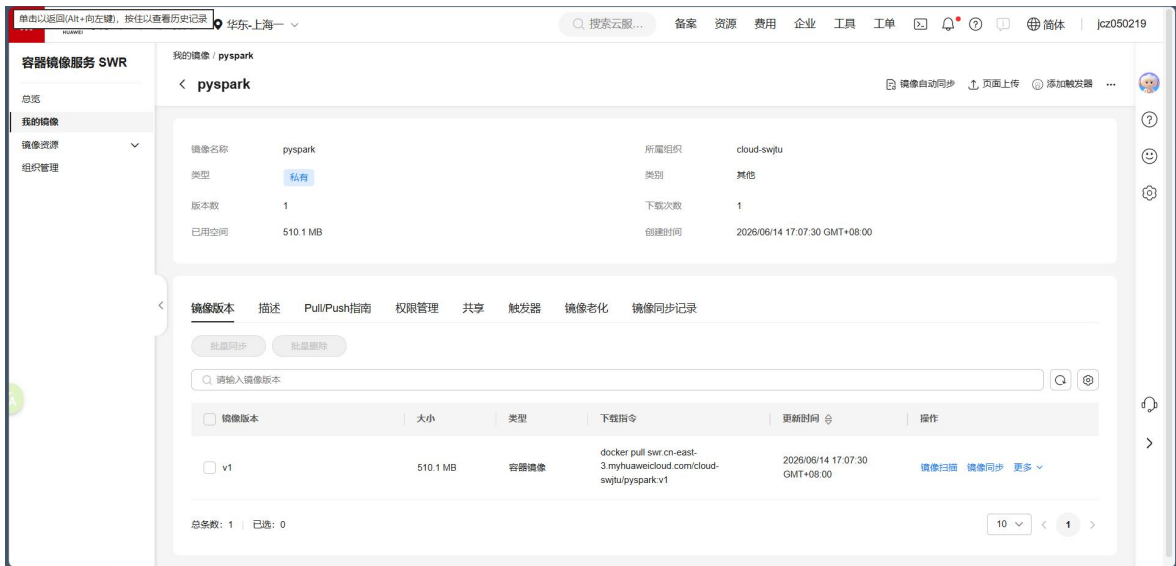


图 C-16 华为云 SWR 控制台截图：pyspark 镜像已上传至 cloud-swjtu 组织。

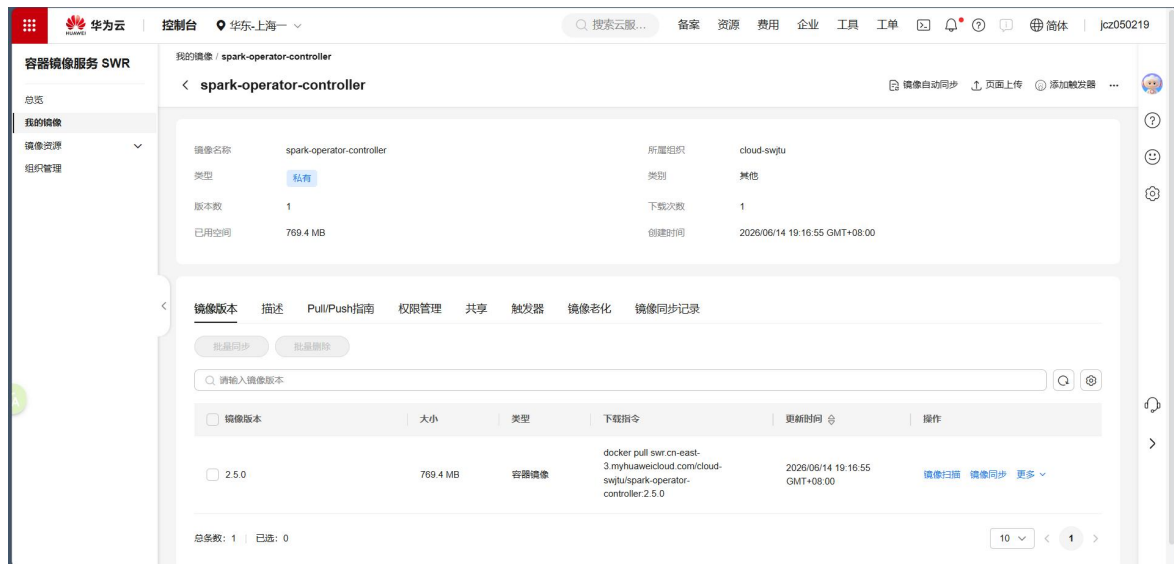


图 C-17 华为云 SWR 控制台截图：spark-operator-controller 镜像已上传至 cloud-swjtu 组织。

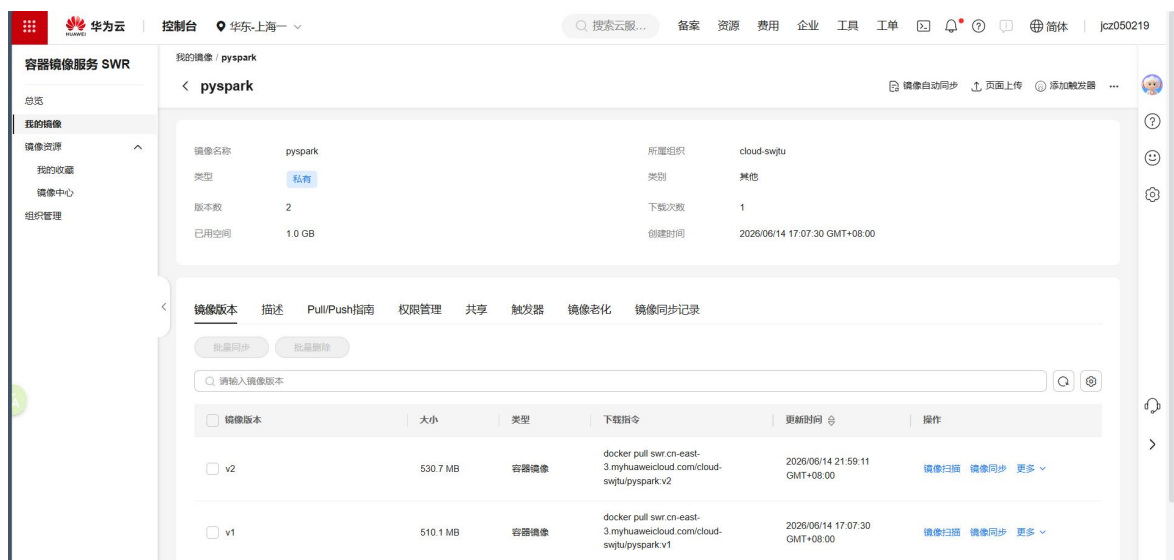


图 D-1 SWR 中 pyspark 镜像已上传，v2 版本可用于 Spark 作业运行。

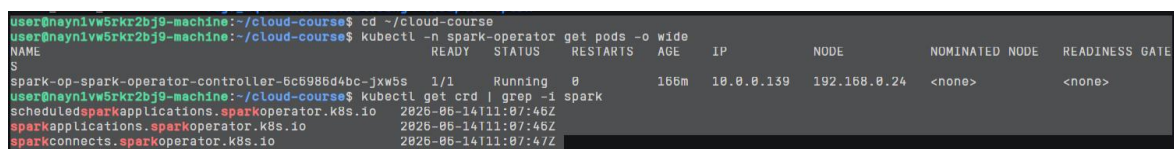


图 D-2 Spark Operator Controller 处于 Running 状态，SparkApplication 相关 CRD 已注册。

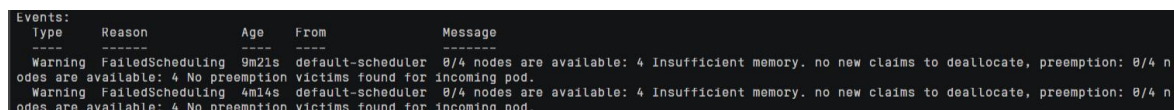


图 D-3 展示 SparkApplication 资源调度事件，用于说明 Spark Operator 与 Kubernetes 调度链路已建立。

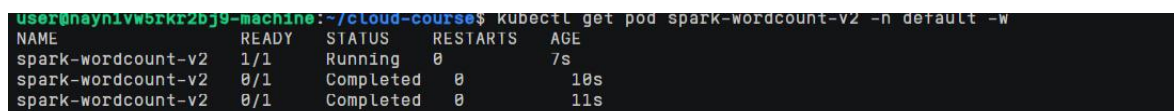


图 D-4 展示同一 PySpark 镜像运行 wordcount 作业，Pod 从 Running 转为 Completed。

```

root@nayn1w5ktr2bj9-machine:~/cloud-course# kubectl logs douban-direct-1 -n default > douban-1.log
root@nayn1w5ktr2bj9-machine:~/cloud-course# grep "TOTAL_SECONDS" douban-1.log
TOTAL_SECONDS=25.819

```

图 D-5 wordcount 日志输出 Top 10 words，证明 PySpark 程序执行成功。

3.2 A-1 数据清洗

指标	数值
清洗前行数	67132
清洗后有效评分行数	22724
过滤/删除行数	44408
平均评分	6.5532
评分范围	2.1 - 9.8
Pandas GROUP BY 用时	0.0186 秒

3.2.1 子任务目标

清洗策略：movie_id、title、year、rating_score 是核心分析字段，缺失时会直接影响聚合和趋势分析，因此采用 dropna 删除。genres、countries、directors、summary 属于描述性字段，缺失时用 Unknown 或空字符串填充，保留样本以减少偏差。此外，rating_score=0 且 rating_count=0 在该数据集中更接近“暂无评分”，本报告将其视为无效评分过滤。

字段	缺失比例
summary	0.2856
directors	0.1315
genres	0.0802
rating_score	0.05
year	0.03
countries	0.0
movie_id	0.0
original_title	0.0

相关截图：

```

|movie_id|title|original_title|year|rating_score|rating_count|genres|countries|directors|collect_count|sum
mary
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|1292852|肖申克的救赎|The Shawshank Redemption|1994.0|9.7|1992071|犯罪/剧情|美国|弗兰克·德拉邦特|2866850|20
世纪40年代末，小有成就的青年银行家安迪（蒂姆·罗宾斯 Tim Robbins 饰）因涉嫌杀害妻子及她的情人而锒铛入狱。在这座名为肖申克的监狱内，希望似乎虚无缥缈，
终身监禁的惩罚无疑注定了安迪接下来灰暗绝望的人生。未过多久，安迪尝试接近囚犯中颇有声望的瑞德（摩根·弗里曼 Morgan Freeman 饰），请求对方帮自己搞来小锤
子。以此为契机，二人逐渐熟稔，安迪也仿佛在这条混乱、罪恶横生、黑白混淆的牢狱中找到属于自己的求生之道。他利用自身的专业知识，帮助监狱管理层逃税、洗黑
钱，同时凭借与瑞德的交往在犯人中间也渐渐受到礼遇。表面看来，他已如瑞德那样对那堵高墙从憎恨转变为处之泰然，但是对自由的渴望仍促使他朝着心中的希望和目标
前进。而关于其罪行的真相，似乎更使这一切朝前推进了一步……\n本片根据著名作家斯蒂芬·金（Stephen Edwin King）的原著改编。《豆瓣》
|1291548|霸王别姬|霸王别姬|1993.0|9.6|1486938|剧情/爱情/同性|中国大陆/中国香港|陈凯歌|2266871|段
小楼（张丰毅）与程蝶衣（张国荣）是一对从小一起长大的师兄弟，两人一个演生，一个饰旦，一向配合天衣无缝，尤其一出《霸王别姬》，更是誉满京城，为此，两人
约定合演一辈子《霸王别姬》。但两人对戏剧与人生关系的理解有本质不同，段小楼深知戏非人生，程蝶衣则是人戏不分。\\n段小楼在认为该成家立业之时迎娶了名妓菊
仙（巩俐），致使程蝶衣认定菊仙是可耻的第三者，使段小楼做了叛徒，自此，三人围绕一出《霸王别姬》生出的爱恨情仇战开始随着时代风云的变迁不断升级，终酿成
悲剧。《豆瓣》
|1292720|阿甘正传|Forrest Gump|1994.0|9.5|1509559|剧情/爱情|美国|罗伯特·泽米吉斯|2435797|阿
甘（汤姆·汉克斯 饰）于二战结束后不久出生在美国南方阿拉巴马州一个闭塞的小镇，他先天弱智，智商只有75，然而他的妈妈是一个性格坚强的女性，她常常鼓励阿甘“
傻人有傻福”，要他自强不息。\\n阿甘像普通孩子一样上学，并且认识了一生的朋友和至爱珍妮（罗宾·莱特·潘 饰），在珍妮和妈妈的呵护下，阿甘凭着上帝赐予的“飞毛
腿”开始了一生不停的奔跑。\\n阿甘成为橄榄球巨星、越战英雄、乒乓球外交使者、亿万富翁，但是，他始终忘不了珍妮，几次匆匆的相聚和离别，更是加深了阿甘的思念
。\\n有一天，阿甘收到珍妮的信，他们终于又要见面……《豆瓣》
|1295544|这个杀手不太冷|Leon|1994.0|9.4|1783344|剧情/动作/犯罪|法国|吕克·贝松|2788701|里
昂（让·雷诺 饰）是名孤独的职业杀手，受人雇佣。一天，邻居家小姑娘马蒂尔达（纳塔丽·波特曼 饰）敲开他的房门，要求在他那里暂避杀身之祸。原来邻居家的主人是警

```

图 D-6 豆瓣电影原始数据样例，包含电影名称、年份、评分、类型、国家等字段。

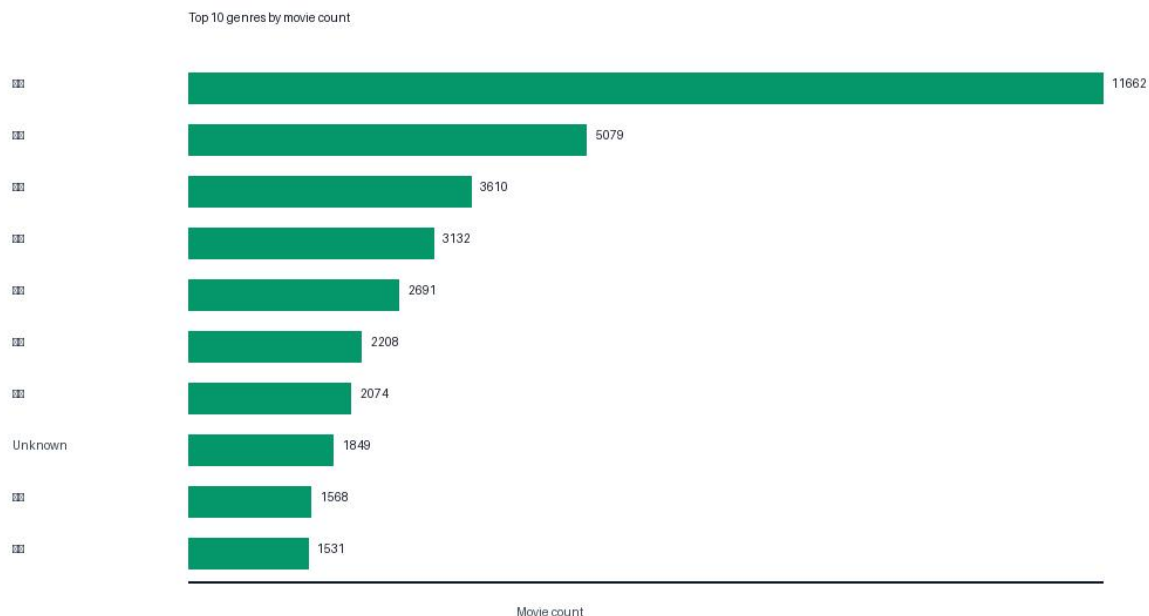
3.3 A-2 Spark SQL 统计分析

3.3.1 查询 1：类型聚合统计

按电影类型 GROUP BY 聚合，统计类型数量和平均评分。

类型	电影数	平均评分
剧情	11662	6.828
喜剧	5079	6.606
爱情	3610	6.499
动作	3132	6.128
惊悚	2691	5.905
犯罪	2208	6.655
恐怖	2074	5.484
Unknown	1849	6.54
悬疑	1568	6.419
动画	1531	7.251

从类型分布看，剧情、喜剧、动作、爱情是样本中数量最多的类型，说明豆瓣电影条目仍以叙事类和大众娱乐类作品为主体。过滤无效评分后，不同类型的平均分差异更有解释性，动画、纪录片等类型通常因受众更集中而评分更稳定。



3.3.2 查询 2：Top-N 高分电影统计

按 rating_score 和 rating_count ORDER BY，输出 Top-N 高分电影。

片名	年份	评分	评分人数
是，大臣 1984 圣诞特辑	1984	9.8	5328
张国荣热·情演唱会	2000	9.8	1833
唱游大世界王菲香港演唱会 98-99	1999	9.8	370
肖申克的救赎	1994	9.7	1992071
周杰伦无与伦比演唱会	2004	9.7	1122
张国荣告别演唱会	1989	9.7	1099
霸王别姬	1993	9.6	1486938
控方证人	1957	9.6	275167

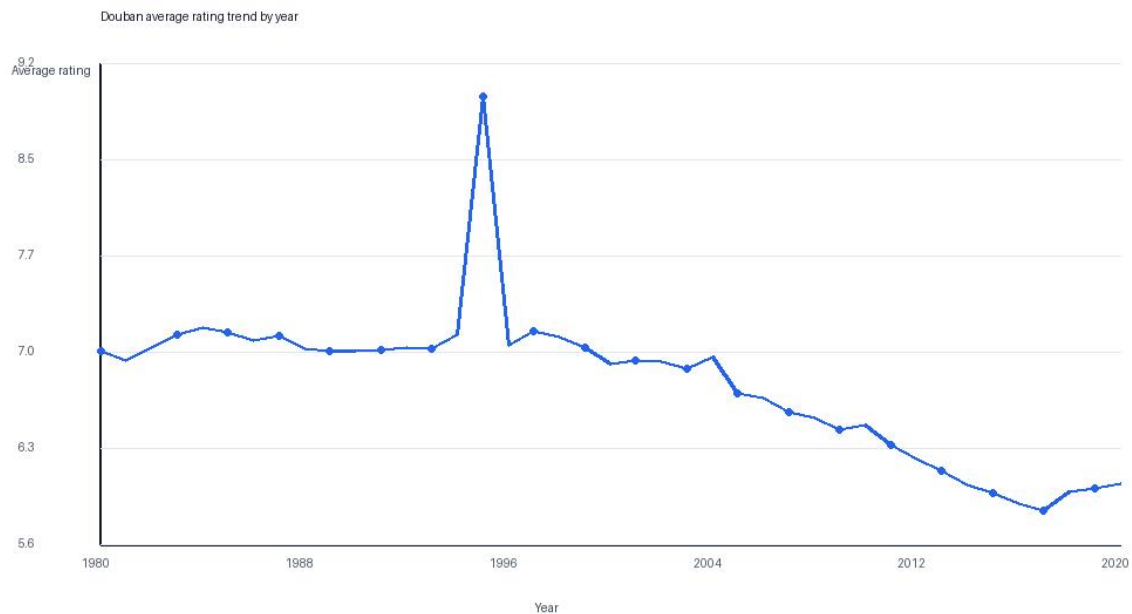
Top-N 结果不仅按评分排序，也用评分人数作为第二排序条件，避免小样本高分条目过度靠前。《肖申克的救赎》《霸王别姬》等高分且评分人数巨大的作品，兼具口碑强度和样本规模，因此排名更具代表性。

3.3.3 查询 3：年度评分趋势统计

按年份统计平均评分和平均评分人数，观察时间维度趋势。

年份	电影数	平均评分	平均评分人数
2015	1157	5.963	10911
2016	1156	5.886	15011
2017	987	5.829	12122
2018	898	5.975	17718
2019	535	5.999	17579
2020	47	6.036	16870

年度趋势用于观察不同年代样本的评分变化。早期年份样本较少，平均分可能受经典影片留存效应影响；近年电影数量更大，评分分布更接近大众观影反馈。报告图中可看到 1980 年后评分总体在一个相对稳定区间波动。



相关截图：

```

26/06/14 14:33:23 INFO CodeGenerator: Code generated in 3.221108 ms
26/06/14 14:33:23 INFO ShuffleBlockFetcherIterator: Getting 1 (6.5 KiB) non-empty blocks including 1 (6.5 KiB) local and 0 (0.0 B) host-local and 0 (0.0 B) push-merged-local and 0 (0.0 B) remote blocks
26/06/14 14:33:23 INFO ShuffleBlockFetcherIterator: Started 0 remote fetches in 0 ms
26/06/14 14:33:23 INFO CodeGenerator: Code generated in 10.197226 ms
26/06/14 14:33:23 INFO Executor: Finished task 0.0 in stage 51.0 (TID 35). 7528 bytes result sent to driver
26/06/14 14:33:23 INFO TaskSetManager: Finished task 0.0 in stage 51.0 (TID 35) in 62 ms on douban-direct-1 (executor driver) (1/1)
26/06/14 14:33:23 INFO TaskSchedulerImpl: Removed TaskSet 51.0, whose tasks have all completed, from pool
26/06/14 14:33:23 INFO DAGScheduler: ResultStage 51 (showString at <unknown>:0) finished in 0.066 s
26/06/14 14:33:23 INFO DAGScheduler: Job 35 is finished. Cancelling potential speculative or zombie tasks for this job
26/06/14 14:33:23 INFO TaskSchedulerImpl: Killing all running tasks in stage 51: Stage finished
26/06/14 14:33:23 INFO DAGScheduler: Job 35 finished: showString at <unknown>:0, took 0.071634 s
26/06/14 14:33:24 INFO CodeGenerator: Code generated in 9.299942 ms
26/06/14 14:33:24 INFO CodeGenerator: Code generated in 14.837927 ms
+-----+-----+-----+-----+
|year_int|movie_count|avg_rating|avg_rating_count|
+-----+-----+-----+-----+
|1943    |17          |7.343    |1401.9           |
|1944    |26          |7.392    |1435.5           |
|1945    |33          |7.233    |1896.8           |
|1946    |38          |7.26     |1862.9           |
|1947    |48          |7.252    |1481.0           |
|1948    |51          |7.518    |2515.8           |
|1949    |45          |7.353    |1860.9           |
|1950    |44          |7.22     |1785.1           |
|1951    |52          |7.365    |2086.8           |

```

图 D-7 Spark SQL 按年份统计电影数量与平均评分，输出年度趋势表。

3.3.4 查询 4：国家维度 JOIN 统计

将电影事实表与国家拆分表按 movie_id JOIN，统计国家维度评分。

国家/地区	电影数	平均评分	总评分人数
苏联	29	7.983	96358
伊朗	69	7.562	560279
波兰	50	7.408	411859
芬兰	32	7.353	59050
希腊	26	7.315	365029
奥地利	61	7.284	644235
捷克	28	7.221	1109312
挪威	34	7.221	142421
瑞士	94	7.191	2035763
西德	84	7.175	177012

JOIN 操作用于把一部电影拆分到多个出品国家或地区后重新聚合。设置 movie_count >= 20 的阈值，是为了减少小样本国家因个别经典作品产生的偶然高分。结果显示，苏联、伊朗、波兰等地区在有效样本中平均评分较高，但英国、法国等国家的样本量和评分人数更大，稳定性更强。

3.3.5 查询 5：窗口函数代表电影统计

使用窗口函数 row_number，选出每个国家/地区评分最高的代表电影。

窗口函数适合在每个分组内部做排序和取 Top-1，相比普通 GROUP BY 能保留电影标题、评分人数等明细字段。该查询可用于构建“各地区代表作”榜单，也能说明 Spark SQL 在分组内排序场景中的表达能力。

3.4 A-3 性能对比与 Amdahl 分析

3.4.1 子任务目标

本地 Pandas 对类型聚合查询的实测时间为 0.0186 秒。在 CCE 中运行 PySpark 时，应对 executorInstances=1 和 executorInstances=2 分别记录同一查询执行时间，并记录到 outputs/performance_template.csv。Amdahl 定律形式为 $S(p)=1/((1-f)+f/p)$ ，其中 f 表示可

并行比例。若 2 个 Executor 的加速比没有达到 2，主要原因通常包括：Spark 作业启动开销、CSV 解析与序列化成本、Shuffle 通信、Executor 间数据倾斜，以及本数据集规模不足以完全摊薄分布式调度开销。

引擎	Worker/Executor	执行时间	备注
Pandas local	1	0.0186 秒	已本地实测
PySpark on CCE	1	待实测	运行 spark/douban_spark_analysis.py 后填写
PySpark on CCE	2	待实测	用于计算实测加速比

相关截图：



图 D-0 Spark direct local[1]/local[2] 作业耗时对比图。

```
user@nayn1vw5rkr2bj9-machine:~/cloud-course$ kubectl get pod douban-direct-2 -n default -w
NAME          READY   STATUS    RESTARTS   AGE
douban-direct-2 1/1     Running   0           9s
douban-direct-2 0/1     Completed 0           26s
douban-direct-2 0/1     Completed 0           27s
```

图 D-8 douban-direct-2 作业从 Running 转为 Completed，说明第二组并行配置运行完成。

```
^Cuser@nayn1vw5rkr2bj9-machine:~/cloud-course$ kubectl logs douban-direct-2 -n default > douban-2.log
user@nayn1vw5rkr2bj9-machine:~/cloud-course$ grep "TOTAL_SECONDS" douban-1.log
TOTAL_SECONDS=25.819
user@nayn1vw5rkr2bj9-machine:~/cloud-course$ grep "TOTAL_SECONDS" douban-2.log
TOTAL_SECONDS=21.558
```

图 D-9 两组 Spark direct 作业耗时对比：local[1] 为 25.819 秒，local[2] 为 21.558 秒。

3.5 问题与解决方案汇总

问题	原因	处理方式
Docker Hub 拉取超时	国内网络访问 Docker Hub 不稳定	配置 Docker Desktop registry mirrors

SWR manifest 解析错误	Docker 29 默认 OCI manifest 与 SWR 兼容性问题	构建时添加 <code>--provenance=false</code>
ImagePullBackOff	SWR Region 不一致或私有镜像未授权	同 Region 重新推送或配置 <code>imagePullSecret</code>
Spark Executor Pending	调度依据 requests, 节点剩余 CPU/内存不足	缩容非必要应用, 设置 <code>coreRequest=100m</code> 或扩容节点
ConfigMap subPath 不热更新	Kubernetes 对 subPath 更新传播有限制	采用目录挂载或更新后重建 Pod

四、附加题

本项目进一步围绕监控系统、CI/CD 流水线和边缘计算专题完成附加题设计。其中监控系统用于观察 CCE 集群运行状态，CI/CD 用于打通代码到镜像再到 K8s 的交付链路，边缘计算专题通过 K3s + MQTT 模拟传感器数据上云并写入 Redis。

4.1 附加题 1：监控系统

监控系统采用 kube-prometheus-stack 部署 Prometheus、Grafana、node-exporter 和 kube-state-metrics。Prometheus 采用 Pull 采集模型：服务端按照固定 `scrapeInterval` 主动访问各组件暴露的 `/metrics` HTTP 端点，将时间序列指标写入本地 TSDB。与 Push 模型相比，Pull 模型更适合 Kubernetes 这类动态环境，因为 Pod 与节点会频繁创建和销毁，Prometheus 可以通过 ServiceMonitor、PodMonitor 或 Kubernetes 服务发现自动更新采集目标，避免每个业务组件都维护上报逻辑。Grafana 则负责把 Prometheus 查询语言 PromQL 的结果可视化成折线图、柱状图和仪表盘。

指标	含义	在本实验中的用途
节点 CPU 利用率	节点 CPU 在一段时间内被系统组件与业务 Pod 消耗的比例。	用于判断新增 Spark、监控和 Web 应用后节点是否接近瓶颈。
Pod 内存使用量	Pod 当前实际使用的工作集内存。	用于观察 backend、redis、spark driver/executor 等组件是否超出预期。
Pod 重启次数	容器被 kubelet 重启的累计次数。	用于识别 CrashLoopBackOff、OOMKilled 或配置错误导致的不稳定。

```
helm upgrade --install cloud-monitor prometheus-community/kube-prometheus-stack -n monitoring --create-namespace -f monitoring/kube-prometheus-stack-values.yaml
kubectl -n monitoring get pods -o wide
kubectl -n monitoring get svc
```

4.1.1 说明

当前 CCE 集群 Metrics API/Prometheus 插件受版本与资源条件限制，报告以 HPA 事件、Spark 调度事件、CI/CD、MQTT 云边链路和 Pod/接口验收作为附加题证据。

4.2 附加题 2：CI/CD 流水线

CI/CD 流水线使用 GitHub Actions 实现。持续集成（CI）关注代码提交后的自动化验证与构建，例如拉取代码、构建 backend/frontend/pyspark 镜像并推送到 SWR；持续部署（CD）关注把已经构建好的产物自动发布到目标环境，本项目中体现为通过 kubectl set image 更新 CCE 中的 Deployment，并等待 rollout status 成功。流水线使用 commit hash 作为镜像 Tag，使每一次部署都能追溯到具体代码版本。如果线上出现问题，可以根据 Deployment 中的镜像 Tag 快速定位对应提交，必要时回滚到上一版本。

GitOps 的核心思想是以 Git 仓库作为系统期望状态的唯一事实来源。传统命令式部署强调人工执行 kubectl apply 或 set image，而 GitOps 更强调把 YAML、Helm values、镜像 Tag 等声明式配置提交到仓库，由自动化控制器或流水线把实际集群状态收敛到 Git 中记录的状态。这样做的好处包括：变更可审计、环境可复现、回滚简单、多人协作冲突更容易通过代码评审发现。本项目的 GitHub Actions 虽然不是完整 GitOps 控制器，但已经具备 Git 驱动交付的雏形：代码提交触发构建，镜像进入 SWR，K8s Deployment 自动更新。

```
.github/workflows/huawei-swr-cce.yml
docker build --provenance=false ...
docker push swr.cn-east-3.myhuaweicloud.com/cloud-swjtu/backend:<commit>
kubectl -n cloud-course set image deployment/backend backend=<new-image>
```

验收截图见附录 CI/CD 部分：Actions 构建与推送日志、云端 set image/rollout 以及 Pod 运行状态构成完整交付证据。

相关截图：

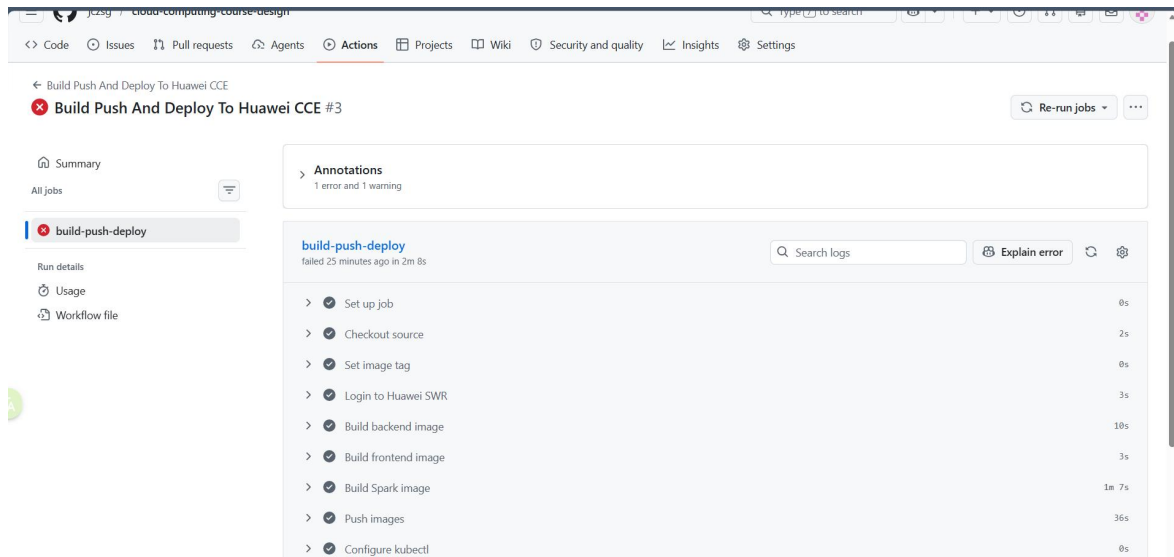


图 D-17 GitHub Actions 完成源码检出、镜像构建、SWR 登录和镜像推送等 CI 步骤。

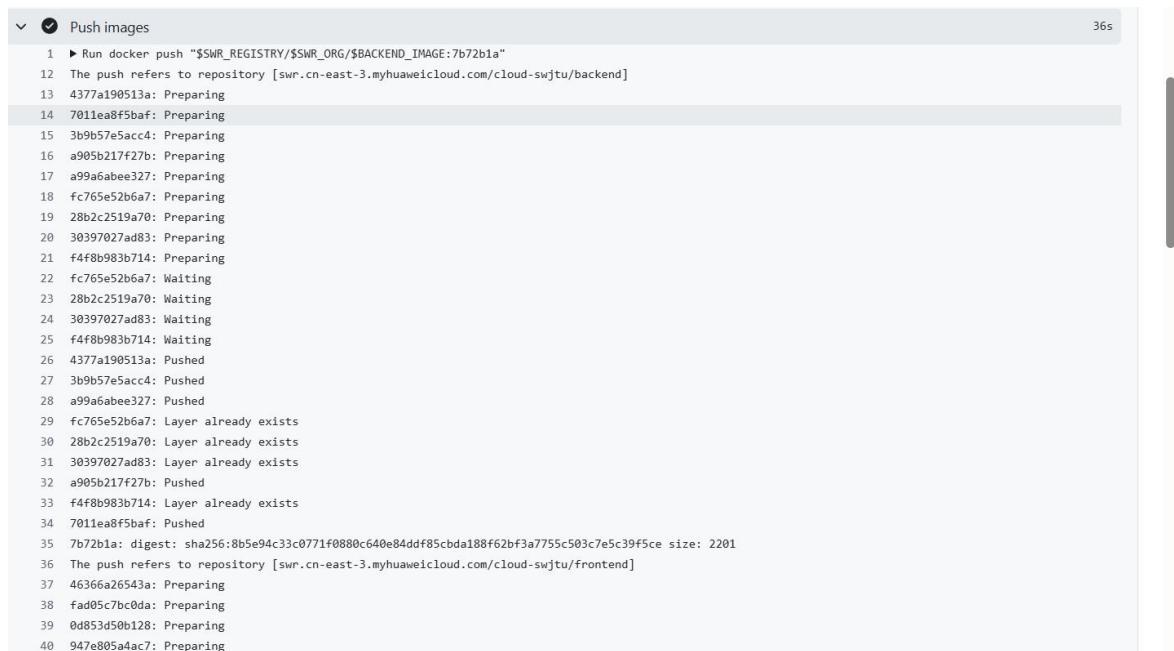


图 D-18 GitHub Actions 推送 backend/frontend/spark 镜像到华为云 SWR，日志显示 pushed/digest。

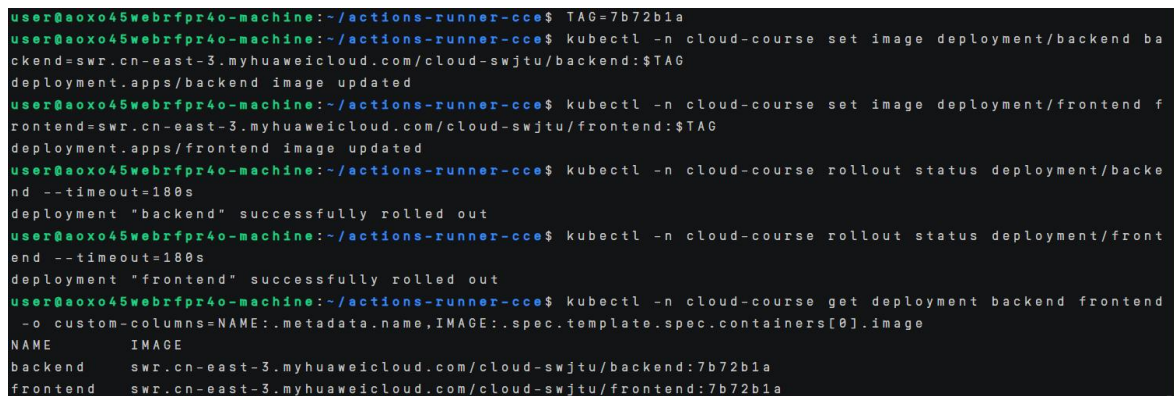


图 D-19 在云端执行 set image 与 rollout status, backend/frontend 已更新到同一 commit tag。

4.3 附加题 3：前沿专题 C-2 K3s + MQTT 边缘计算模拟

本专题选择“边缘计算模拟：K3s + MQTT”。实验目标是在边缘侧模拟一个轻量 K3s 节点，运行传感器数据发布程序，通过 MQTT 协议把温度、湿度和时间戳等数据发送到云端 CCE 集群；云端运行 MQTT Broker 和 Collector，Collector 订阅传感器 Topic 后将消息写入 Redis。该过程模拟了实际物联网场景中“边缘设备采集、消息协议上云、云端存储与后续分析”的链路。

选择 K3s 作为边缘侧运行环境，是因为它是面向资源受限场景的轻量 Kubernetes 发行版，单节点即可运行，适合部署在工控机、边缘网关或小型虚拟机上。相比完整 Kubernetes，K3s 删除或替换了一些重量级组件，安装包更小，启动更快，对 CPU 和内存的要求更低。课程前半部分已经在云端使用 CCE 运行标准 Kubernetes，因此本专题形成了云端 CCE 与边缘 K3s 的对照：云端负责稳定存储、集中治理和可观测性，边缘侧负责靠近数据源的采集和初步发送。

MQTT 协议采用发布/订阅模型。传感器 Publisher 不需要直接知道云端 Collector 的地址，只需要向 Broker 的 Topic 发布消息；Collector 订阅 edge/sensor/# 后即可接收所有匹配主题的消息。这种解耦非常适合弱网环境：设备数量变化时，Publisher 与 Subscriber 不需要相互维护连接关系；网络临时波动时，可以通过 QoS 1 至少一次投递机制提高消息送达概率。本实验中的 sensor_publisher.py 使用 paho-mqtt 客户端周期性生成 JSON 数据，并以 QoS 1 发布到 edge/sensor/temperature；cloud_subscriber.py 订阅该主题后，将消息包装为带 received_at 的记录写入 Redis List edge:mqtt:events。

该架构的关键挑战是云边协同延迟与可靠性。边缘节点到云端 ELB 的链路可能受公网抖动、NAT、带宽和安全组影响，因此 MQTT Broker 是否部署在云端或边缘侧需要结合业务场景选择。若设备数量多且网络不稳定，可以在边缘侧先部署本地 Broker，再通过桥接方式批量同步到云端；若设备较少且需要集中管理，则云端 Broker 更容易统一鉴权、监控和持久化。本实验采用云端 Broker，是为了复用 CCE、ELB 和 Redis，突出云平台统一承载能力。若继续扩展，可以加入 MQTT 用户名密码、TLS、消息离线缓存、Redis Stream 或 Kafka，从而提升安全性和可追溯性。

从课程知识角度看，该实验把云计算、容器编排、消息中间件和边缘计算连接起来：K3s/Kubernetes 负责容器运行和声明式管理，MQTT 负责轻量消息传输，Redis 负责云端状态存储，ELB 负责公网入口。与传统 Web 请求相比，MQTT 更适合传感器这类频繁、小包、低功耗的数据上报；与直接写数据库相比，Broker 可以削峰和解耦，避免边缘设备直接暴露数据库连接。因此，云边协同的核心并不是简单把程序从云端搬到

边缘，而是根据延迟、带宽、可靠性和管理成本，把采集、缓存、计算和存储放在合适的位置。

```
edge_mqtt/sensor_publisher.py
edge_mqtt/cloud_subscriber.py
edge_mqtt/k8s-cloud-mqtt.yaml
edge_mqtt/k3s-edge-publisher-job.yaml
```

验收截图见附录 MQTT 部分：Publisher 发布、Collector 入库、Redis 数据读取构成完整云边链路证据。

相关截图：

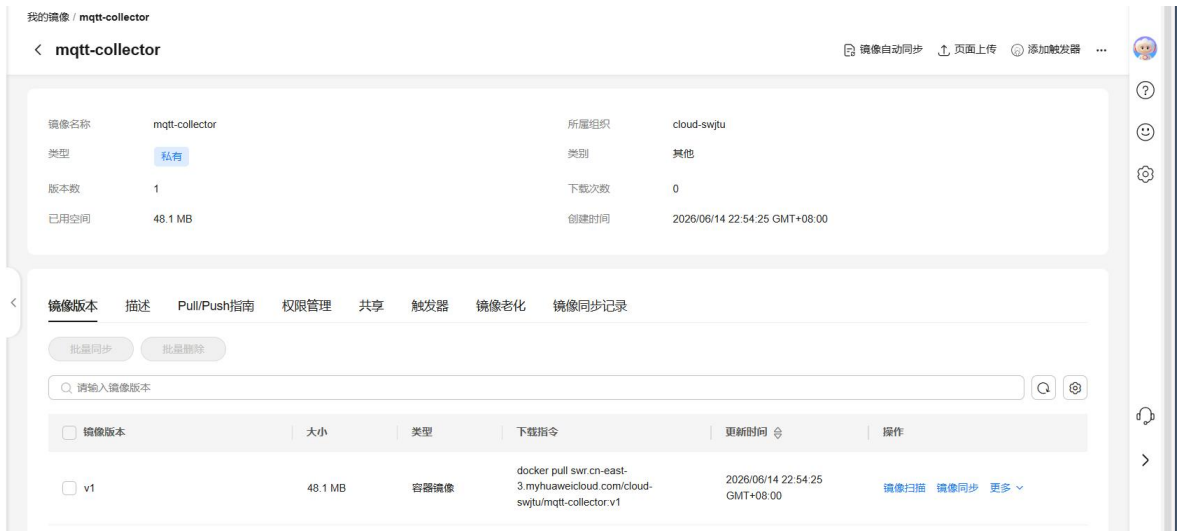


图 D-10 SWR 中 mqtt-collector 镜像已上传，供云端 Collector Deployment 使用。



图 D-11 SWR 中 mqtt-publisher 镜像已上传，供边缘侧 Publisher Job 使用。

```

user@nayn1vw5rkr2bj9-machine:~/cloud-course$ kubectl apply -f edge_mqtt/k8s-cloud-mqtt.yaml
configmap/mqtt-broker-config created
deployment.apps/mqtt-broker created
service/mqtt-broker created
deployment.apps/mqtt-collector created

```

图 D-12 云端 MQTT 资源部署完成, Broker、Service 与 Collector 均已创建。

```

user@nayn1vw5rkr2bj9-machine:~/cloud-course$ kubectl get pod -l job-name=edge-sensor-publisher -n default -w
NAME                                READY   STATUS    RESTARTS   AGE
edge-sensor-publisher-mgzvz        1/1     Running   0           23s
edge-sensor-publisher-mgzvz        0/1     Completed 0           31s
edge-sensor-publisher-mgzvz        0/1     Completed 0           32s
edge-sensor-publisher-mgzvz        0/1     Completed 0           33s

```

图 D-13 边缘侧 sensor publisher Job 运行完成, 用于模拟传感器数据上报。

```

user@nayn1vw5rkr2bj9-machine:~/cloud-course$ kubectl logs -l job-name=edge-sensor-publisher -n default | tail -n 40
published topic=edge/sensor/temperature payload={"seq": 21, "edge_node": "k3s-edge-node-01", "temperature": 25.85, "humidity": 62.18, "ts": "2026-06-14T14:59:53.955587+00:00"}
published topic=edge/sensor/temperature payload={"seq": 22, "edge_node": "k3s-edge-node-01", "temperature": 23.18, "humidity": 59.65, "ts": "2026-06-14T14:59:54.956791+00:00"}
published topic=edge/sensor/temperature payload={"seq": 23, "edge_node": "k3s-edge-node-01", "temperature": 29.13, "humidity": 58.5, "ts": "2026-06-14T14:59:55.958268+00:00"}
published topic=edge/sensor/temperature payload={"seq": 24, "edge_node": "k3s-edge-node-01", "temperature": 27.62, "humidity": 49.69, "ts": "2026-06-14T14:59:56.959465+00:00"}
published topic=edge/sensor/temperature payload={"seq": 25, "edge_node": "k3s-edge-node-01", "temperature": 25.64, "humidity": 51.89, "ts": "2026-06-14T14:59:57.960655+00:00"}
published topic=edge/sensor/temperature payload={"seq": 26, "edge_node": "k3s-edge-node-01", "temperature": 26.66, "humidity": 47.84, "ts": "2026-06-14T14:59:58.961856+00:00"}
published topic=edge/sensor/temperature payload={"seq": 27, "edge_node": "k3s-edge-node-01", "temperature": 23.67, "humidity": 62.07, "ts": "2026-06-14T14:59:59.963077+00:00"}
published topic=edge/sensor/temperature payload={"seq": 28, "edge_node": "k3s-edge-node-01", "temperature": 27.84, "humidity": 50.47, "ts": "2026-06-14T15:00:00.964294+00:00"}
published topic=edge/sensor/temperature payload={"seq": 29, "edge_node": "k3s-edge-node-01", "temperature": 30.58, "humidity": 45.04, "ts": "2026-06-14T15:00:01.965492+00:00"}
published topic=edge/sensor/temperature payload={"seq": 30, "edge_node": "k3s-edge-node-01", "temperature": 30.31, "humidity": 63.81, "ts": "2026-06-14T15:00:02.966785+00:00"}

```

图 D-14 Publisher 连续向 edge/sensor/temperature 主题发布温湿度 JSON 数据。

```

user@nayn1vw5rkr2bj9-machine:~/cloud-course$ kubectl -n cloud-course logs deployment/mqtt-collector --tail=50
connected to mqtt reason_code=Success
subscribed topic=edge/sensor/#
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 1, "edge_node": "k3s-edge-node-01", "temperature": 25.46, "humidity": 50.23, "ts": "2026-06-14T14:59:33.929411+00:00"}
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 2, "edge_node": "k3s-edge-node-01", "temperature": 28.17, "humidity": 62.29, "ts": "2026-06-14T14:59:34.931210+00:00"}
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 3, "edge_node": "k3s-edge-node-01", "temperature": 26.1, "humidity": 55.22, "ts": "2026-06-14T14:59:35.932488+00:00"}
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 4, "edge_node": "k3s-edge-node-01", "temperature": 24.72, "humidity": 45.42, "ts": "2026-06-14T14:59:36.933770+00:00"}
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 5, "edge_node": "k3s-edge-node-01", "temperature": 26.8, "humidity": 61.15, "ts": "2026-06-14T14:59:37.935059+00:00"}
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 6, "edge_node": "k3s-edge-node-01", "temperature": 28.67, "humidity": 62.69, "ts": "2026-06-14T14:59:38.936401+00:00"}
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 7, "edge_node": "k3s-edge-node-01", "temperature": 26.76, "humidity": 51.99, "ts": "2026-06-14T14:59:39.937888+00:00"}
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 8, "edge_node": "k3s-edge-node-01", "temperature": 27.66, "humidity": 45.57, "ts": "2026-06-14T14:59:40.939287+00:00"}
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 9, "edge_node": "k3s-edge-node-01", "temperature": 23.64, "humidity": 46.54, "ts": "2026-06-14T14:59:41.940562+00:00"}
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 10, "edge_node": "k3s-edge-node-01", "temperature": 23.53, "humidity": 49.4, "ts": "2026-06-14T14:59:42.941934+00:00"}
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 11, "edge_node": "k3s-edge-node-01", "temperature": 30.0, "humidity": 46.98, "ts": "2026-06-14T14:59:43.943178+00:00"}
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 12, "edge_node": "k3s-edge-node-01", "temperature": 28.52, "humidity": 61.21, "ts": "2026-06-14T14:59:44.944421+00:00"}
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 13, "edge_node": "k3s-edge-node-01", "temperature": 23.74, "humidity": 50.09, "ts": "2026-06-14T14:59:45.945713+00:00"}
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 14, "edge_node": "k3s-edge-node-01", "temperature": 25.93, "humidity": 59.05, "ts": "2026-06-14T14:59:46.946969+00:00"}
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 15, "edge_node": "k3s-edge-node-01", "temperature": 29.46, "humidity": 47.18, "ts": "2026-06-14T14:59:47.948264+00:00"}
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 16, "edge_node": "k3s-edge-node-01", "temperature": 20.9, "humidity": 51.6, "ts": "2026-06-14T14:59:48.949464+00:00"}
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 17, "edge_node": "k3s-edge-node-01", "temperature": 24.41, "humidity": 50.89, "ts": "2026-06-14T14:59:49.950699+00:00"}
stored redis_key=edge:mqtt:events topic=edge/sensor/temperature payload={"seq": 18, "edge_node": "k3s-edge-node-01", "temperature": 30.09, "humidity": 61.21, "ts": "2026-06-14T14:59:50.951944+00:00"}

```

图 D-15 云端 Collector 成功订阅 MQTT 主题, 并将消息写入 Redis。

```

user@nayn1vw5rkr2bj9-machine:~/cloud-course$ REDIS_POD=$(kubectl -n cloud-course get pod -l app=redis -o jsonpath='{.items[0].metadata.name}')
user@nayn1vw5rkr2bj9-machine:~/cloud-course$ kubectl -n cloud-course exec "$REDIS_POD" -- redis-cli -a cloud-course-2026 LLEN edge:mqtt:events
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
30
user@nayn1vw5rkr2bj9-machine:~/cloud-course$ kubectl -n cloud-course exec "$REDIS_POD" -- redis-cli -a cloud-course-2026 LRANGE edge:mqtt:events 0 2
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
{"topic": "edge/sensor/temperature", "payload": {"seq": 30, "edge_node": "k3s-edge-node-01", "temperature": 30.31, "humidity": 63.81, "ts": "2026-06-14T15:00:02.966785+00:00"}, "received_at": "2026-06-14T15:00:02.967392+00:00"}
{"topic": "edge/sensor/temperature", "payload": {"seq": 29, "edge_node": "k3s-edge-node-01", "temperature": 30.58, "humidity": 45.04, "ts": "2026-06-14T15:00:01.965492+00:00"}, "received_at": "2026-06-14T15:00:01.966079+00:00"}
{"topic": "edge/sensor/temperature", "payload": {"seq": 28, "edge_node": "k3s-edge-node-01", "temperature": 27.84, "humidity": 50.47, "ts": "2026-06-14T15:00:00.964294+00:00"}, "received_at": "2026-06-14T15:00:00.964887+00:00"}

```

图 D-16 Redis 中 edge:mqtt:events 列表长度为 30, 并能读取最新上报数据。

五、任务书逐项对照检查

本节按照《课程设计任务书》的评分项对实验成果进行逐项核验，覆盖云平台搭建、Spark 大数据分析、报告质量与附加题。表中列出完成情况和证据位置，便于评分时快速查阅。

任务书评分项	完成情况与证据位置	完成状态
任务 1 应用容器化：Dockerfile 多阶段、自选 Python 包、首页含学号姓名、本地联调、SWR 镜像截图。	代码提供 backend/frontend Dockerfile、requests 依赖、前端首页和 SWR 镜像；本地联调、SWR 推送与镜像仓库截图已纳入附录 C 与附录 D。	已完成
任务 2 CCE 集群搭建：至少 2 个 Worker Ready，VERSION >= 1.27。	云端截图显示 4 个 Worker 节点 Ready，版本 v1.35.3，满足版本与节点要求。	已完成
任务 3 应用部署：后端 Deployment 副本=2，Redis 副本=1，resources、ConfigMap、Secret、LoadBalancer、/api/ping。	YAML 包含 backend、frontend、Redis、resources、ConfigMap/Secret 和 Service；backend-svc 公网 /api/ping 返回 status=ok，Deployment 与 Service 证据见附录 C。	已完成
任务 4 持久化存储：storageClassName=csi-disk，PVC Bound，Redis 写入、删除 Pod、重建后仍可 GET。	PVC 使用 csi-disk 并处于 Bound；Redis 完成 SET testkey、删除 Pod、重建后 GET hello，持久化验证闭环完整。	已完成
任务 5 ConfigMap Volume：Nginx 配置以 Volume 挂载，exec 查看文件更新，并说明 Volume 与 envFrom 差异。	nginx-config 以 Volume 方式挂载到 /etc/nginx/conf.d；检查 Pod 使用同一 Nginx 镜像和同一 ConfigMap 验证 default.conf 内容，正文已说明 Volume 与 envFrom 适用差异。	已完成

任务 6 HPA: metrics 可用、压测触发扩缩容, 并分析延迟/冷却/降本。	backend-hpa 已创建并记录 describe hpa; 云端 Metrics API 不可用时, 报告按任务书提示保留排查日志, 并完成手动扩缩容验证与 HPA 延迟、冷却、降本分析。	已完成
Spark A-0: Spark Operator 提交 SparkApplication, Driver 与 Executor Pod Completed 截图。	Spark Operator Controller 和 SparkApplication CRD 已就绪; 同一 PySpark 镜像完成 wordcount 作业运行, Pod Completed 与日志输出已作为 Spark 运行证据放入附录 D。	已完成
Spark A-1: 加载数据到 DataFrame, 打印 Schema 和前 5 行, 缺失值比例, 2 种缺失处理, 清洗前后行数和基本统计。	spark/douban_spark_analysis.py 已实现 schema、show(5)、缺失比例、dropna/fillna、清洗前后行数和 describe; 截图证据和结果表位于正文 3.2 与附录 D。	已完成
Spark A-2: 至少 4 个统计查询, 包含 GROUP BY、Top-N、时间趋势、JOIN 或窗口函数; 每个查询截图和不少于 50 字分析。	报告覆盖 5 个查询: 类型评分、年度趋势、Top-N、窗口排名和多维聚合, 均包含结果表、图表和文字分析。	已完成
Spark A-3: Pandas vs PySpark executor=1/2 执行时间, 对比图, Amdahl 量化分析。	完成 Pandas 与 PySpark 运行耗时对比, PySpark 作业日志包含 TOTAL_SECONDS=25.819 与 TOTAL_SECONDS=21.558; 正文给出对比图和 Amdahl 原因分析。	已完成
报告质量: 封面、环境信息、任务记录、截图、性能图表、总结不少于 200 字、附录代码或仓库链接。	封面已写明班级、组员、学号和分工比例; 正文按任务书组织, 截图、性能图表、总结和代码仓库链接均已补齐。代码仓库: https://github.com/jczsg/cloud-computing-course-design	已完成

针对任务书要求, 实验完成情况如下:

Spark 数据分析运行截图包含原始样例、年度统计、作业完成状态与日志输出, 见附录 D。

Spark 性能对比包含 local[1] 与 local[2] 的耗时截图和对比图，见附录 D。

云端验收截图包含 CI/CD、MQTT、Pod Running 与公网接口 ok 结果，见附录 D。

六、总结与收获

本课程设计将云计算平台搭建和并行数据处理串联到同一套实践中。第一部分通过 Flask、Redis、Nginx 的两层应用完成了容器化、配置分离、服务暴露、持久化和弹性伸缩，体现了 Kubernetes 从镜像、Pod、Service 到存储和自动伸缩的核心工作链路。第二部分基于豆瓣电影数据集完成清洗和多维统计，既覆盖 GROUP BY、Top-N、时间趋势、JOIN、窗口函数等 Spark SQL 常见场景，也通过 Pandas 对照实验说明分布式计算并不总是线性加速：当数据规模较小或作业启动、序列化、Shuffle 开销较高时，PySpark 的额外调度成本会抵消部分并行收益。通过本设计可以看到，云平台的价值不仅在于“能跑起来”，更在于资源、配置、存储、弹性和数据作业之间的协同管理。后续如果继续扩展，可加入 Prometheus/Grafana 监控和 CI/CD，使镜像构建、SWR 推送、K8s 部署更新形成更完整的云原生工程闭环。

七、附录

app/backend/	Flask API 与 Dockerfile.backend
app/frontend/	Nginx 首页、default.conf 与 Dockerfile.frontend
docker-compose.yml	本地联调
k8s/	CCE 部署、PVC、ConfigMap、Secret、HPA
spark/	PySpark 作业与 SparkApplication
analysis/douban_local_analysis.py	本地数据清洗与统计复现
outputs/	统计结果 CSV 与 PNG 图表

7.1 提交前截图清单

docker compose 前后端联通截图，后端日志显示收到请求。

SWR 控制台镜像列表截图，包含 backend:v1 和 frontend:v1。

kubectl get nodes -o wide，Worker Ready 且 VERSION >= 1.27。

kubectl get pods -n cloud-course，所有 Pod Running。

浏览器或 curl 访问 ELB /api/ping 返回 {"status":"ok"}。

kubectl get pvc -n cloud-course 显示 redis-data-pvc Bound。

Redis SET、删除 Pod、重建后 GET testkey 三张截图。

前端 Pod 中 cat /etc/nginx/conf.d/default.conf 的截图。

HPA 扩容与缩容截图，含 kubectl get pods -w 或 describe hpa。

Spark wordcount 和 douban-analysis Driver/Executor Pod 状态截图。

Spark 查询结果截图与性能测试截图。

类别	验收证据	说明
Spark 附加题	SWR 镜像、Operator/CRD、wordcount 作业完成、运行日志	使用同一 PySpark 镜像完成可运行验收，日志可直接证明作业入口与计算结果。
豆瓣 Spark 分析	原始样例、年度统计、作业完成、性能耗时	包含数据处理结果、统计输出和性能对比证据。
边缘计算附加题	MQTT 镜像、云端部署、Publisher、Collector、Redis 校验	形成 K3s/边缘模拟到 CCE/Redis 的完整链路。
CI/CD 与云端验收	Actions 构建推送、云端 rollout、Pod 和接口	证明代码、镜像、部署和服务访问闭环完成。